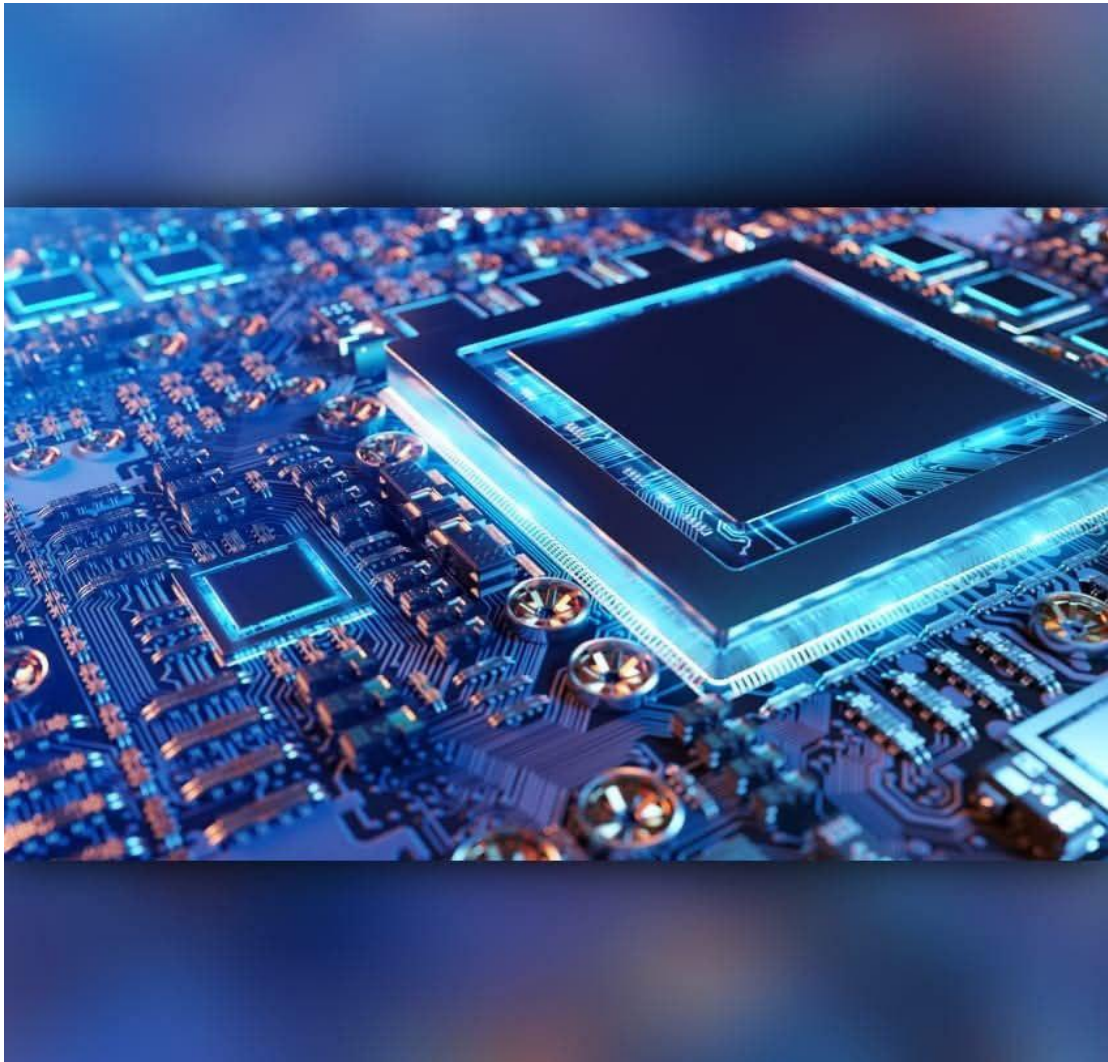


Digital Electronics Design

Assignment_Two

Prepared by:

Omar Ahmed Abdelaty Mohammed



Submitted to: Eng. Kareem Waseem

Question 1

Code



```
1  module alu #(parameter N = 4, parameter OPCODE = 0) (  
2      input  [N-1:0] in0,  
3      input  [N-1:0] in1,  
4      output reg [N-1:0] out  
5  );  
6  
7  always @(*) begin  
8      case (OPCODE)  
9          2'b00: out = in0 + in1;  
10         2'b01: out = in0 | in1;  
11         2'b10: out = in0 - in1;  
12         2'b11: out = in0 ^ in1;  
13         default: out = 0;  
14     endcase  
15 end  
16  
17 endmodule
```

```

1  module alu_tb_add;
2
3      reg [3:0] in0, in1;
4      wire [3:0] out;
5      integer i;
6
7      alu #(.N(4), .OPCODE(0)) dut (.in0(in0), .in1(in1), .out(out));
8
9      initial begin
10         for (i = 0; i < 10; i = i + 1) begin
11             in0 = $random;
12             in1 = $random;
13             #10;
14             if (out !== in0 + in1)
15                 $display("FAIL: %d + %d != %d", in0, in1, out);
16         end
17         $stop;
18     end
19
20 endmodule

```

Wave - Default											
	Msgs										
in0	4'hd	4	9	d	5	1	6	d	9	5	
in1	4'hd	1	3	d	2	d		c	6	a	7
out	4'ha	5	c	a	7	e	3	9	f		c
i	32'h00000002	00000000	00000001	00000002	00000003	00000004	00000005	00000006	00000007	00000008	00000009
in0	4'hd	4	9	d	5	1	6	d	9	5	
in1	4'hd	1	3	d	2	d		c	6	a	7
out	4'ha	5	c	a	7	e	3	9	f		c
i	32'h00000002	00000000	00000001	00000002	00000003	00000004	00000005	00000006	00000007	00000008	00000009

```

1  module alu_tb_sub;
2
3      reg [3:0] in0, in1;
4      wire [3:0] out;
5      integer i;
6
7      alu #(.N(4), .OPCODE(2)) dut (.in0(in0), .in1(in1), .out(out));
8
9      initial begin
10         for (i = 0; i < 10; i = i + 1) begin
11             in0 = $random;
12             in1 = $random;
13             #10;
14             if (out !== in0 - in1)
15                 $display("FAIL: %d - %d != %d", in0, in1, out);
16         end
17         $stop;
18     end
19
20 endmodule

```

Wave - Default

	Msgs										
in0	4'h0	4	9	d	5	1	6	d	9	5	
in1	4'h0	1	3	d	2	d		c	6	a	7
out	4'h0	3	6	0	3	4	9	1	3	b	e
i	32'h00000002	00000000	00000001	00000002	00000003	00000004	00000005	00000006	00000007	00000008	00000009

```

1  module alu_tb_or;
2
3      reg [3:0] in0, in1;
4      wire [3:0] out;
5      integer i;
6
7      alu #(.N(4), .OPCODE(1)) dut (.in0(in0), .in1(in1), .out(out));
8
9      initial begin
10         for (i = 0; i < 10; i = i + 1) begin
11             in0 = $random;
12             in1 = $random;
13             #10;
14             if (out !== (in0 | in1))
15                 $display("FAIL: %b | %b != %b", in0, in1, out);
16         end
17         $stop;
18     end
19
20 endmodule

```

Wave - Default											
	Msgs										
in0	4h5	4	9	d	5	1	6	d	9	5	
in1	4h7	1	3	d	2	d	c	6	a	7	
out	4h7	5	b	d	7	d	f	d	f	7	
i	32'h0000000a	00000000	00000001	00000002	00000003	00000004	00000005	00000006	00000007	00000008	00000009

```

1  module alu_tb_xor;
2
3      reg [3:0] in0, in1;
4      wire [3:0] out;
5      integer i;
6
7      alu #(.N(4), .OPCODE(3)) dut (.in0(in0), .in1(in1), .out(out));
8
9      initial begin
10         for (i = 0; i < 10; i = i + 1) begin
11             in0 = $random;
12             in1 = $random;
13             #10;
14             if (out !== (in0 ^ in1))
15                 $display("FAIL: %b ^ %b != %b", in0, in1, out);
16         end
17         $stop;
18     end
19
20 endmodule

```

Wave - Default										
	Msgs									
in0	4'h9	4	9	d	5	1	6	d	9	5
in1	4'h3	1	3	d	2	d		c	6	a
out	4'ha	5	a	0	7	c	b	1	f	2
i	32'h00000001	00000000	00000001	00000002	00000003	00000004	00000005	00000006	00000007	00000008

Question 2

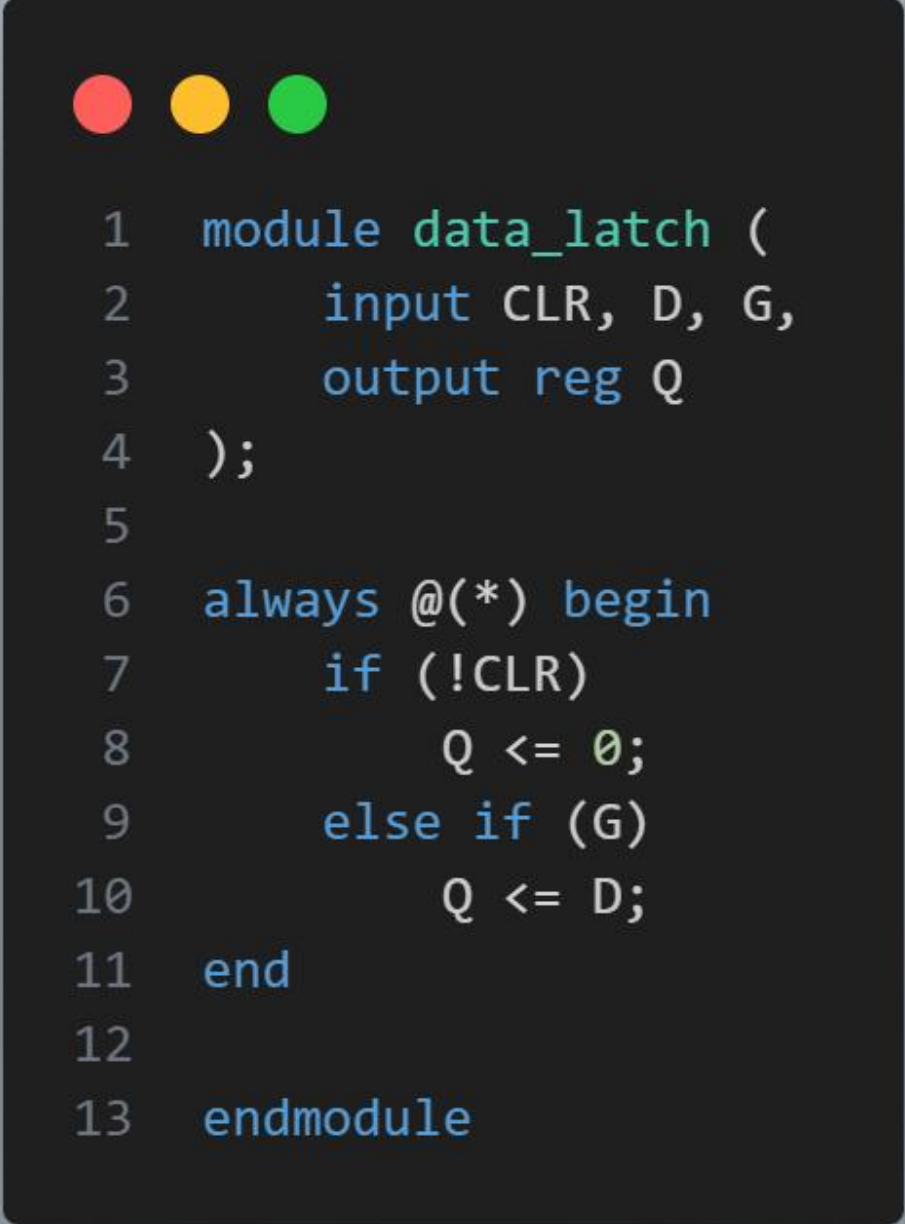
Code



```
1  module alu_sync #(parameter N = 4, parameter OPCODE = 0) (  
2      input clk,  
3      input rst,  
4      input [N-1:0] in0,  
5      input [N-1:0] in1,  
6      output reg [N-1:0] out  
7  );  
8  
9      reg [N-1:0] result;  
10  
11     always @(*) begin  
12         case (OPCODE)  
13             2'b00: result = in0 + in1;  
14             2'b01: result = in0 | in1;  
15             2'b10: result = in0 - in1;  
16             2'b11: result = in0 ^ in1;  
17             default: result = 0;  
18         endcase  
19     end  
20  
21     always @(posedge clk) begin  
22         if (rst)  
23             out <= 0;  
24         else  
25             out <= result;  
26         end  
27  
28     endmodule
```

Question 3

Code



```
1  module data_latch (  
2      input CLR, D, G,  
3      output reg Q  
4  );  
5  
6  always @(*) begin  
7      if (!CLR)  
8          Q <= 0;  
9      else if (G)  
10         Q <= D;  
11  end  
12  
13  endmodule
```

Question 4

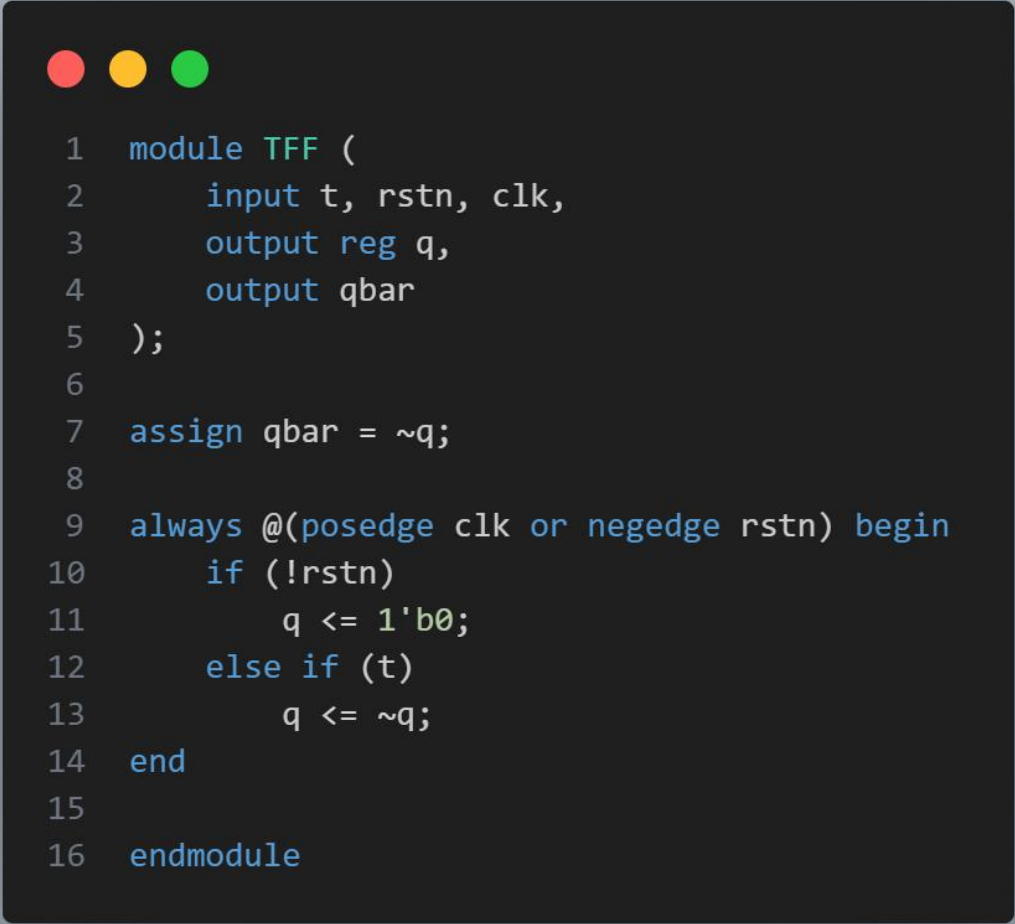
Code



```
1  module latch #(parameter LAT_WIDTH = 8) (  
2      input aset,gate,aclr,  
3      input [LAT_WIDTH-1:0] data,  
4      output reg [LAT_WIDTH-1:0] q  
5  );  
6  
7  always @(*) begin  
8      if (aclr)  
9          q = 0;  
10     else if (aset)  
11         q = {LAT_WIDTH{1'b1}};  
12     else if (gate)  
13         q = data;  
14 end  
15  
16 endmodule
```

Question 5a

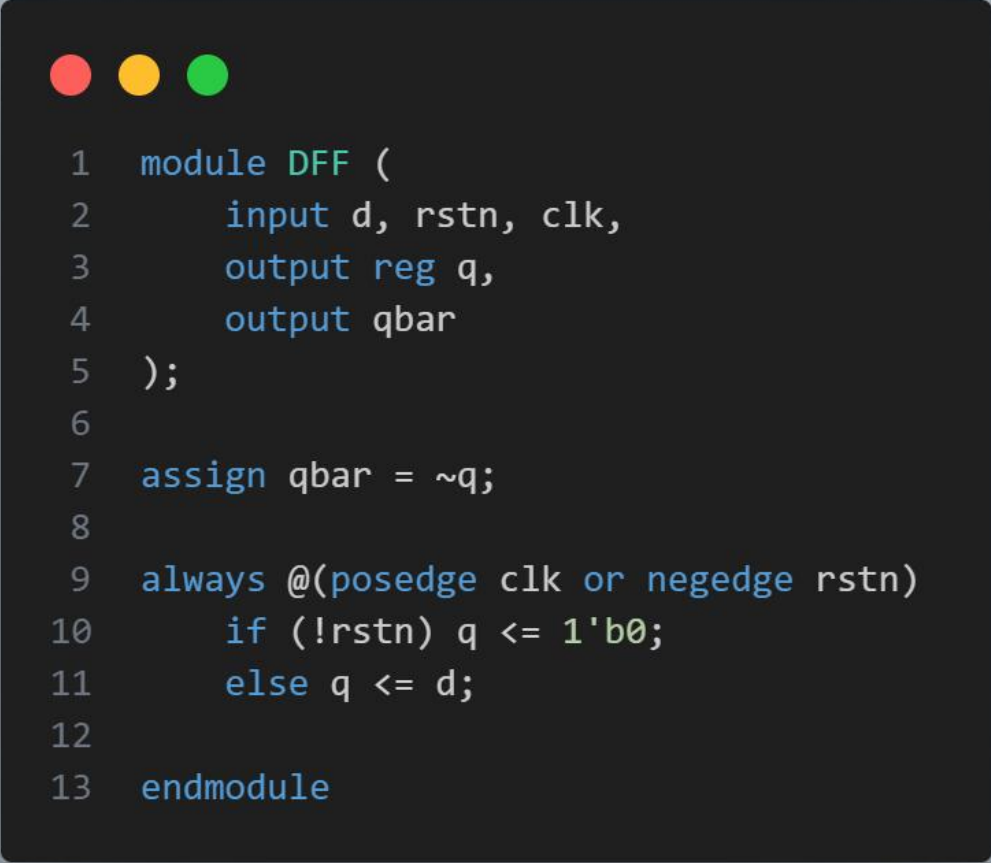
Code



```
1  module TFF (  
2      input t, rstn, clk,  
3      output reg q,  
4      output qbar  
5  );  
6  
7  assign qbar = ~q;  
8  
9  always @(posedge clk or negedge rstn) begin  
10      if (!rstn)  
11          q <= 1'b0;  
12      else if (t)  
13          q <= ~q;  
14  end  
15  
16  endmodule
```

Question 5b

Code



```
1  module DFF (  
2      input d, rstn, clk,  
3      output reg q,  
4      output qbar  
5  );  
6  
7  assign qbar = ~q;  
8  
9  always @(posedge clk or negedge rstn)  
10      if (!rstn) q <= 1'b0;  
11      else q <= d;  
12  
13  endmodule
```

Question 5c

Code



```
1  module FlipFlop #(
2      parameter FF_TYPE = "DFF"
3  ) (
4      input d, rstn, clk,
5      output reg q,
6      output qbar
7  );
8
9  assign qbar = ~q;
10
11  always @(posedge clk or negedge rstn) begin
12      if (!rstn)
13          q <= 1'b0;
14      else if (FF_TYPE == "DFF")
15          q <= d;
16      else if (FF_TYPE == "TFF")
17          q <= d ? ~q : q;
18  end
19  end
20
21  endmodule
```